

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 8
«Графы»

Выполнил работу
Дышлевский Игорь
Академическая группа №J3111
Принято
Ментор, Владислав Вершинин

Санкт-Петербург
2024

1. Введение

Цель: реализовать DFS, BFS, алгоритм Дейкстры и A* на графе города.

Задачи:

- Реализовать работу с графом из текстового файла
- Реализовать структуру вершины (Node) и графа (Graph)
- Написать Алгоритм поиска в глубину (далее DFS)
- Написать Алгоритм поиска в ширину (далее BFS)
- Написать Алгоритм Дейкстры
- Написать A*
- Посчитать время работы алгоритмов
- Сравнить методы

2. Теоретическая подготовка

Типы данных:

- Int - целые числа
- Double - десятичные дроби с повышенной точностью
- String - строки
- Bool - true/false
- Vector - вектор
- Pair - пара
- Map - словарь (сопоставляет элементу значение)
- Queue - очередь (первым вошел - первым вышел. Оптимизирована для некоторых задач)
- Priority_queue - упорядоченная очередь (очередь с упорядоченными элементами)

Алгоритмы:

- DFS

DFS - это алгоритм обхода в глубину, который идёт «вглубь» графа пока может, если «ветка» закончилась он поднимается на несколько уровней и продолжает с в другом направлении. DFS не используется для поиска расстояния из-за того, что он может прийти в вершину по любому пути, в

котором она есть (то есть может пойти и по самому длинному, главное, чтобы она там была), из-за чего в любых графах, кроме деревьев (потому что там только один путь из любой в любую вершину) он будет выдавать значения, которые в большинстве случаев будут неверными. Основным применением алгоритма является топологическая сортировка, проверка на циклы, проверка связности.

- BFS

BFS - это алгоритм обзора графа в ширину, который равномерно расходуется от изначальной вершины во всевозможных направлениях (шагает по уровням отдаления от стартовой вершины). Этот алгоритм можно при создании игра (его частный случай - волновой алгоритм), для нахождения минимального пути в невзвешенных графах, для обхода деревьев по уровням (далее я буду его применять от стартовой до конечной вершины, но граф взвешенный - его результатом будет минимальный по количеству ребер путь).

- Дейкстра

Алгоритм Дейкстры работает по схожему с BFS образу, но тут дополнительной появляется список расстояний от стартовой вершины до текущей и если можно прийти в уже посещенную вершину с меньшими затратами Дейкстра перезапишет расстояние от стартовой до текущей вершины на новое, тем самым находя оптимальный путь от стартовой до конечной вершины. Он применяется во взвешенных графах с неотрицательными ребрами для поиска лучшего (по затратам) пути от стартовой вершины до всех.

- A*

A* - это модифицированный алгоритм Дейкстры, который предназначен для поиска хорошего (не во всех случаях оптимального - зависит от правильности оценки (эвристики)) пути от стартовой до конечной вершины. За счет понимания, которой вершины необходимо достичь, происходит оптимизация связанная с добавлением эвристики, которая описывает расстояние от текущей вершины до конечной и используется для отсекаания очевидно (с точки зрения эвристики) плохих вариантов, за счет чего у

распространения алгоритма появляется направление в конечную вершину (а не во все стороны, как в случае с Дейкстрой), что зачастую экономит время для поиска пути. Самое наглядное его применение - навигационные системы.

3. Реализация

- Функции для восстановления пути и расстояния (DFS и BFS не предназначены для подсчета расстояния - в них важен только путь, из-за этого выделена отдельная функция).

```
vector<pair<Node*, double>> find_path_and_len(map<Node*, Node*>& parent, map<Node*, double>& edges_cost, Node* start, Node* goal) {
    vector<Node*> path;
    Node* current = goal;
    while (current != start) {
        path.push_back(current);
        current = parent[current];
    }
    path.push_back(start);
    reverse(path.begin(), path.end());
    vector<pair<Node*, double>> result;
    for (int i = 0; i < path.size(); i++) {
        result.push_back({path[i], edges_cost[path[i]]});
    }
    return result;
}

vector<Node*> find_path(map<Node*, Node*>& parent, Node* start, Node* goal) {
    vector<Node*> path;
    Node* current = goal;
    while (current != start) {
        path.push_back(current);
        current = parent[current];
    }
    path.push_back(start);
    reverse(path.begin(), path.end());
    return path;
}
```

Рисунок 3.1 Функции для восстановления пути и расстояния

- DFS

```
void DFS_Recursive(map<Node*, bool>& visited, Node* start, map<Node*, Node*>& parent) {
    visited[start] = true;
    for (pair<Node*, double> neighbor : start->nodes) {
        if (!visited[neighbor.first]) {
            parent[neighbor.first] = start;
            DFS_Recursive(visited, neighbor.first, parent);
        }
    }
}

vector<Node*> DFS(Node* start, Node* goal) {
    map<Node*, bool> visited;
    map<Node*, Node*> parent;
    parent[start] = nullptr;
    for (Node* node : nodes) {
        visited[node] = false;
    }
    DFS_Recursive(visited, start, parent);
    return find_path(parent, start, goal);
}
```

Рисунок 3.2 Обход в Глубину

- BFS

```
vector<Node*> BFS(Node* start, Node* goal) {
    map<Node*, bool> visited;
    map<Node*, Node*> parent;
    parent[start] = nullptr;
    for (Node* node : nodes) {
        visited[node] = false;
    }
    queue<Node*> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        Node* v = q.front();
        q.pop();
        for (pair<Node*, double> u : v->nodes) {
            if (!visited[u.first]) {
                parent[u.first] = v;
                visited[u.first] = true;
                q.push(u.first);
            }
        }
    }
    return find_path(parent, start, goal);
}
```

Рисунок 3.3 Обход в Ширину

- Дейкстра

```
vector<pair<Node*, double>> Dijkstra(Node* start, Node* goal) {
    map<Node*, double> dist;
    map<Node*, Node*> parent;
    map<Node*, double> edges_cost;
    parent[start] = nullptr;
    edges_cost[start] = 0;
    for (Node* node : nodes) {
        dist[node] = numeric_limits<double>::infinity();
    }
    dist[start] = 0;
    priority_queue<pair<double, Node*>, vector<pair<double, Node*>>, greater<>> pq;
    pq.emplace(0, start);
    while (!pq.empty()) {
        pair<double, Node*> u = pq.top();
        pq.pop();
        if (dist[u.second] < u.first) {
            continue;
        }
        for (pair<Node*, double> v : u.second->nodes) {
            double new_dist = dist[u.second] + v.second;
            if (new_dist < dist[v.first]) {
                parent[v.first] = u.second;
                edges_cost[v.first] = v.second;
                dist[v.first] = new_dist;
                pq.emplace(new_dist, v.first);
            }
        }
    }
    return find_path_and_len(parent, edges_cost, start, goal);
}
```

Рисунок 3.4 Алгоритм Дейкстры

- A*

```
vector<pair<Node*, double>> Astar(Node* start, Node* goal) {
    map<Node*, double> dist;
    map<Node*, double> f;
    map<Node*, Node*> parent;
    map<Node*, double> edges_cost;
    map<Node*, bool> closed;
    parent[start] = nullptr;
    edges_cost[start] = 0;
    for (Node* node : nodes) {
        dist[node] = numeric_limits<double>::infinity();
        f[node] = numeric_limits<double>::infinity();
        closed[node] = false;
    }
    dist[start] = 0;
    f[start] = euclidean_distance({start->lat, start->lon}, {goal->lat, goal->lon});
    priority_queue<pair<double, Node*>, vector<pair<double, Node*>>, greater<>> pq;
    pq.emplace(f[start], start);
    while (!pq.empty()) {
        pair<double, Node*> u = pq.top();
        pq.pop();
        if (u.second == goal) {
            return find_path_and_len(parent, edges_cost, start, goal);
        }
        if (closed[u.second]) {
            continue;
        }
        closed[u.second] = true;
        for (pair<Node*, double> v : u.second->nodes) {
            if (closed[v.first]) {
                continue;
            }
            double new_dist = dist[u.second] + v.second;
            if (new_dist < dist[v.first]) {
                parent[v.first] = u.second;
                edges_cost[v.first] = v.second;
                dist[v.first] = new_dist;
                f[v.first] = euclidean_distance({v.first->lat, v.first->lon}, {goal->lat, goal->lon}) + ne
w_dist;
                pq.emplace(f[v.first], v.first);
            }
        }
    }
    return find_path_and_len(parent, edges_cost, start, goal);
}
```

Рисунок 3.5 A*

4. Результаты и время работы

```
347333
Start: 59.884937 30.3677841
End: 59.9443927 30.2953995

DFS
DFS Path Len: 29184
DFS Time: 0.884423

BFS
BFS Path Len: 105
BFS Time: 0.986601

Dijkstra
Dijkstra Path Len: 206
Dijkstra Dist: 0.1129958603
Dijkstra Time: 2.092925

Astar
Astar Path Len: 206
Astar Dist: 0.1129958603
Astar Time: 0.726925
```

Рисунок 4.1 Вывод программы

Выводы из результатов:

1. A* работает быстрее Дейкстры
2. A* находит верное расстояние
3. BFS находит более короткий путь по количеству ребер, но более длинный по расстоянию

5. Заключение

В ходе работы были реализованы DFS, BFS, алгоритм Дейкстры и A*.

6. Приложения

ПРИЛОЖЕНИЕ А

Листинг кода файла main8.cpp

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <utility>
#include <algorithm>
#include <utility>
#include <ctime>
#include <queue>

using namespace std;

string strip(const string& s, const string& chars = " \t\n\r\f\v")
{
    size_t start = 0;
    while (start < s.size() && chars.find(s[start]) !=
string::npos)
    {
        start++;
    }
    size_t end = s.size();
    while (end > start && chars.find(s[end - 1]) != string::npos)
    {
        end--;
    }
    return s.substr(start, end - start);
}

vector<string> split(const string& s, const string& delimiter = "
")
{
    vector<string> result;
    size_t start = 0;
    while (true)
```

```

    {
        size_t pos = s.find(delimiter, start);
        if (pos == string::npos)
        {
            result.push_back(s.substr(start));
            break;
        }
        result.push_back(s.substr(start, pos - start));
        start = pos + delimiter.size();
    }
    return result;
}

string replace_char(const string& s, char oldchar, char newchar)
{
    string result = s;
    for (auto &ch : result) {
        if (ch == oldchar) {
            ch = newchar;
        }
    }
    return result;
}

double euclidean_distance(const pair<double, double>& a, const
pair<double, double>& b) {
    double dx = a.first - b.first;
    double dy = a.second - b.second;
    return sqrt(dx*dx + dy*dy);
}

struct Node {
    double lon, lat;
    vector<pair<Node*, double>> nodes;
};

struct Graph {
    vector<Node*> nodes;
    map<pair<double, double>, Node*> pair2node;

    Node* find_closest_node(double lat, double lon) {
        double min_dist = numeric_limits<double>::infinity();
        Node* min_node = nullptr;
        for (auto &node : nodes) {

```

```

        double dist = euclidean_distance({node->lat, node->lon}, {lat, lon});
        if (dist < min_dist) {
            min_dist = dist;
            min_node = node;
        }
    }
    return min_node;
}

Node* add_node(double lat, double lon) {
    Node* node = new Node();
    node->lat = lat;
    node->lon = lon;
    nodes.push_back(node);
    return node;
}

void read_graph(string filename) {
    ifstream fin(filename);
    if (!fin) {
        cerr << "cannot open file with graph\n";
    }
    string line;
    while (getline(fin, line)) {
        if (!line.empty() && line.back() == ';') {
            line.pop_back();
        }
        replace(line.begin(), line.end(), ':', ';');
        line = strip(line);
        line = replace_char(line, ';', ' ');
        vector<string> tokens = split(line);
        vector<string> tmp = split(tokens[0], ",");
        pair<double, double> from = {stod(tmp[1]),
stod(tmp[0])};
        Node* parent_node;
        if (pair2node.count(from)) {
            parent_node = pair2node[from];
        } else {
            parent_node = add_node(stod(tmp[1]),
stod(tmp[0]));
            pair2node[from] = parent_node;
        }
        for (int i = 1; i < tokens.size(); i++) {
            vector<string> tmp = split(tokens[i], ",");

```

```

        pair<double, double> to = {stod(tmp[1]),
stod(tmp[0])});
        Node* child_node;
        if (pair2node.count(to)) {
            child_node = pair2node[to];
        } else {
            child_node = add_node(stod(tmp[1]),
stod(tmp[0]));
            pair2node[to] = child_node;
        }
        parent_node->nodes.emplace_back(child_node,
stod(tmp[2]));
        child_node->nodes.emplace_back(parent_node,
stod(tmp[2]));
    }
}

vector<pair<Node*, double>> find_path_and_len(map<Node*,
Node*>& parent, map<Node*, double>& edges_cost, Node* start, Node*
goal) {
    vector<Node*> path;
    Node* current = goal;
    while (current != start) {
        path.push_back(current);
        current = parent[current];
    }
    path.push_back(start);
    reverse(path.begin(), path.end());
    vector<pair<Node*, double>> result;
    for (int i = 0; i < path.size(); i++) {
        result.push_back({path[i], edges_cost[path[i]]});
    }
    return result;
}

vector<Node*> find_path(map<Node*, Node*>& parent, Node*
start, Node* goal) {
    vector<Node*> path;
    Node* current = goal;
    while (current != start) {
        path.push_back(current);
        current = parent[current];
    }
    path.push_back(start);

```

```

        reverse(path.begin(), path.end());
        return path;
    }

    void DFS_Recursive(map<Node*, bool>& visited, Node* start,
map<Node*, Node*>& parent) {
        visited[start] = true;
        for (pair<Node*, double> neighbor : start->nodes) {
            if (!visited[neighbor.first]) {
                parent[neighbor.first] = start;
                DFS_Recursive(visited, neighbor.first, parent);
            }
        }
    }

    vector<Node*> DFS(Node* start, Node* goal) {
        map<Node*, bool> visited;
        map<Node*, Node*> parent;
        parent[start] = nullptr;
        for (Node* node : nodes) {
            visited[node] = false;
        }
        DFS_Recursive(visited, start, parent);
        return find_path(parent, start, goal);
    }

    vector<Node*> BFS(Node* start, Node* goal) {
        map<Node*, bool> visited;
        map<Node*, Node*> parent;
        parent[start] = nullptr;
        for (Node* node : nodes) {
            visited[node] = false;
        }
        queue<Node*> q;
        q.push(start);
        visited[start] = true;
        while (!q.empty()) {
            Node* v = q.front();
            q.pop();
            for (pair<Node*, double> u : v->nodes) {
                if (!visited[u.first]) {
                    parent[u.first] = v;
                    visited[u.first] = true;
                    q.push(u.first);
                }
            }
        }
    }

```

```

    }
    }
    return find_path(parent, start, goal);
}

vector<pair<Node*, double>> Dijkstra(Node* start, Node* goal)
{
    map<Node*, double> dist;
    map<Node*, Node*> parent;
    map<Node*, double> edges_cost;
    parent[start] = nullptr;
    edges_cost[start] = 0;
    for (Node* node : nodes) {
        dist[node] = numeric_limits<double>::infinity();
    }
    dist[start] = 0;
    priority_queue<pair<double, Node*>, vector<pair<double,
Node*>>, greater<>> pq;
    pq.emplace(0, start);
    while (!pq.empty()) {
        pair<double, Node*> u = pq.top();
        pq.pop();
        if (dist[u.second] < u.first) {
            continue;
        }
        for (pair<Node*, double> v : u.second->nodes) {
            double new_dist = dist[u.second] + v.second;
            if (new_dist < dist[v.first]) {
                parent[v.first] = u.second;
                edges_cost[v.first] = v.second;
                dist[v.first] = new_dist;
                pq.emplace(new_dist, v.first);
            }
        }
    }
    return find_path_and_len(parent, edges_cost, start, goal);
}

vector<pair<Node*, double>> Astar(Node* start, Node* goal) {
    map<Node*, double> dist;
    map<Node*, double> f;
    map<Node*, Node*> parent;
    map<Node*, double> edges_cost;
    map<Node*, bool> closed;

```



```

    parent[start] = nullptr;
    edges_cost[start] = 0;
    for (Node* node : nodes) {
        dist[node] = numeric_limits<double>::infinity();
        f[node] = numeric_limits<double>::infinity();
        closed[node] = false;
    }
    dist[start] = 0;
    f[start] = euclidean_distance({start->lat, start->lon},
{goal->lat, goal->lon});
    priority_queue<pair<double, Node*>, vector<pair<double,
Node*>>, greater<>> pq;
    pq.emplace(f[start], start);
    while (!pq.empty()) {
        pair<double, Node*> u = pq.top();
        pq.pop();
        if (u.second == goal) {
            return find_path_and_len(parent, edges_cost,
start, goal);
        }
        if (closed[u.second]) {
            continue;
        }
        closed[u.second] = true;
        for (pair<Node*, double> v : u.second->nodes) {
            if (closed[v.first]) {
                continue;
            }
            double new_dist = dist[u.second] + v.second;
            if (new_dist < dist[v.first]) {
                parent[v.first] = u.second;
                edges_cost[v.first] = v.second;
                dist[v.first] = new_dist;
                f[v.first] = euclidean_distance({v.first->lat,
v.first->lon}, {goal->lat, goal->lon}) + new_dist;
                pq.emplace(f[v.first], v.first);
            }
        }
    }
    return find_path_and_len(parent, edges_cost, start, goal);
}

};

```

```

int main() {
    cout << setprecision(10);
    Graph graph;
    graph.read_graph("spb_graph.txt");
    cout << graph.nodes.size() << "\n";

    Node* start = graph.find_closest_node(59.884690, 30.367749);
    Node* end = graph.find_closest_node(59.944134, 30.295449);

    cout << "Start: " << start->lat << " " << start->lon << "\n";
    cout << "End: " << end->lat << " " << end->lon << "\n\n";

    cout << "DFS\n";
    time_t time1 = clock();
    vector<Node*> path = graph.DFS(start, end);
    time_t time2 = clock();
    cout << "DFS Path Len: " << path.size() << "\n";
    cout << "DFS Time: " << (long double)(time2 - time1) / (long
double)(CLOCKS_PER_SEC) << "\n\n";

    cout << "BFS\n";
    time_t time3 = clock();
    path = graph.BFS(start, end);
    time_t time4 = clock();
    cout << "BFS Path Len: " << path.size() << "\n";
    cout << "BFS Time: " << (long double)(time4 - time3) / (long
double)(CLOCKS_PER_SEC) << "\n\n";

    cout << "Dijkstra\n";
    time_t time5 = clock();
    vector<pair<Node*, double>> path_and_len =
graph.Dijkstra(start, end);
    time_t time6 = clock();
    double dist = 0;
    for (pair<Node*, double> p : path_and_len) {
        dist += p.second;
    }
    cout << "Dijkstra Path Len: " << path_and_len.size() << "\n";
    cout << "Dijkstra Dist: " << dist << "\n";
    cout << "Dijkstra Time: " << (long double)(time6 - time5) /
(long double)(CLOCKS_PER_SEC) << "\n\n";

    cout << "Astar\n";
    time_t time7 = clock();

```

```

    path_and_len = graph.Astar(start, end);
    time_t time8 = clock();
    dist = 0;
    for (pair<Node*, double> p : path_and_len) {
        dist += p.second;
    }
    cout << "Astar Path Len: " << path_and_len.size() << "\n";
    cout << "Astar Dist: " << dist << "\n";
    cout << "Astar Time: " << (long double)(time8 - time7) / (long
double)(CLOCKS_PER_SEC) << "\n\n";
    return 0;
}

```